Name :- Vrusha Patel

Enrolment Number :- 12402080601155

Batch :- 2D8

Subject :- Programming With Python (202044504)

Topic :- Assignment-1

GITHUB LINK: https://github.com/vrushapatel001-pixel/python-programming

1. Campus Merit Analyzer using Compound Data Structures

```
# Campus Merit Analyzer

# Taking basic input
n = int(input("Enter number of students: "))
k = int(input("Enter number of top students to display: "))
m = int(input("Enter number of subjects: "))

students = []
semesters = {}

# Taking student details
for i in range(n):

    print("\nEnter details of Student", i + 1)

    enrollment = input("Enter enrollment number: ")
    name = input("Enter student name: ")
    semester = int(input("Enter semester: "))
    cpi = float(input("Enter CPI: "))

    marks_input = input(
        "Enter marks for " + str(m) + " subjects separated by spaces: "
    )

    marks = list(map(int, marks_input.split()))

    # Calculate average marks
    average = sum(marks) / m

    student = {
        "enrollment": enrollment,
        "name": name,
        "semester": semester,
        "cpi": cpi,
        "marks": marks,
        "average": average
    }

    students.append(student)

    # Store students semester-wise
    if semester not in semesters:
        semesters[semester] = []

    semesters[semester].append(student)

# Display top K students from each semester
print("\n----- TOP STUDENTS BY SEMESTER ----- ")

for semester in sorted(semesters):

    student_list = semesters[semester]

    # Sort by:
    # 1. Higher CPI
```

```
    # 2. Higher average marks
    # 3. Smaller enrollment number
    student_list.sort(
        key=lambda student: (
            -student["cpi"],
            -student["average"],
            student["enrollment"]
        )
    )

    print("Semester", semester, end=": ")

    for student in student_list[:k]:
        print(student["enrollment"], end=" ")

    print()

# Display subject-wise toppers
print("\n----- SUBJECT-WISE TOPPERS ----- ")

for subject in range(m):

    highest_mark = -1

    # Find highest mark
    for student in students:
        if student["marks"][subject] > highest_mark:
            highest_mark = student["marks"][subject]

    # Find all students having highest mark
    toppers = []

    for student in students:
        if student["marks"][subject] == highest_mark:
            toppers.append(student["enrollment"])

    toppers.sort()

    print("Subject", subject + 1, end=": ")

    for enrollment in toppers:
        print(enrollment, end=" ")

    print()
```

Output :

```
Enter number of students: 2
Enter number of top students to display: 1
Enter number of subjects: 1

Enter details of Student 1
Enter enrollment number: 1
Enter student name: Ram
Enter semester: 2
Enter CPI: 8
Enter marks for 1 subjects separated by spaces: 45

Enter details of Student 2
Enter enrollment number: 2
Enter student name: Shyam
Enter semester: 2
Enter CPI: 9
Enter marks for 1 subjects separated by spaces: 66

----- TOP STUDENTS BY SEMESTER -----
Semester 2: 2

----- SUBJECT-WISE TOPPERS -----
Subject 1: 2
```

2. Optimized Password Audit with Pattern Constraints

```
# Q2. Optimized Password Audit with Pattern Constraints

from collections import deque

# ---------- Trie Node ----------

class TrieNode:
    def __init__(self):
        self.children = {}
        self.fail = 0
        self.is_banned = False

# ---------- Aho-Corasick ----------

class AhoCorasick:

    def __init__(self):
        self.nodes = [TrieNode()]

    def add_word(self, word):
        current = 0

        for ch in word.lower():

            if ch not in self.nodes[current].children:
                self.nodes[current].children[ch] = len(self.nodes)
                self.nodes.append(TrieNode())

            current = self.nodes[current].children[ch]

        self.nodes[current].is_banned = True

    def build(self):

        queue = deque()

        for child in self.nodes[0].children.values():
            queue.append(child)

        while queue:

            current = queue.popleft()

            for ch, child in self.nodes[current].children.items():

                fail_node = self.nodes[current].fail

                while (
                    fail_node != 0
                    and ch not in self.nodes[fail_node].children
                ):
                    fail_node = self.nodes[fail_node].fail

                if ch in self.nodes[fail_node].children:
                    self.nodes[child].fail = (
```

```
                    self.nodes[fail_node].children[ch]
                )
            else:
                self.nodes[child].fail = 0

            if self.nodes[
                self.nodes[child].fail
            ].is_banned:
                self.nodes[child].is_banned = True

            queue.append(child)

    def contains_banned_word(self, password):

        current = 0

        for ch in password.lower():

            while (
                current != 0
                and ch not in self.nodes[current].children
            ):
                current = self.nodes[current].fail

            if ch in self.nodes[current].children:
                current = self.nodes[current].children[ch]
            else:
                current = 0

            if self.nodes[current].is_banned:
                return True

        return False

# ---------------- Password Checking Functions ----------------

def has_required_characters(password):

    has_lower = False
    has_upper = False
    has_digit = False
    has_special = False

    for ch in password:

        if ch.islower():
            has_lower = True

        elif ch.isupper():
            has_upper = True

        elif ch.isdigit():
            has_digit = True

        elif ch in "$#@":
            has_special = True
```

```
    return (
        has_lower
        and has_upper
        and has_digit
        and has_special
    )

def has_repeated_character(password):

    count = 1

    for i in range(1, len(password)):

        if password[i] == password[i - 1]:
            count += 1

            if count > 3:
                return True
        else:
            count = 1

    return False

def classify_password(password, checker):

    # Check length
    if len(password) < 6 or len(password) > 12:
        return "WEAK_LENGTH"

    # Check banned words
    if checker.contains_banned_word(password):
        return "COMPROMISED"

    # Check required characters
    if not has_required_characters(password):
        return "WEAK_PATTERN"

    # Check repeated characters
    if has_repeated_character(password):
        return "WEAK_PATTERN"

    return "STRONG"

# ---------- Main Program ----------

print("----- PASSWORD AUDIT PROGRAM ----- ")

# Number of banned words
b = int(input("\nEnter number of banned words: "))

checker = AhoCorasick()

# Enter banned words
print("\nEnter banned words:")

for i in range(b):
```

```
    word = input(
        "Enter banned word " + str(i + 1) + ": "
    )

    checker.add_word(word)

# Build the Trie
checker.build()

# Number of passwords
n = int(input("\nEnter number of passwords to check: "))

print("\n----- ENTER PASSWORDS -----")

# Check passwords
for i in range(1, n + 1):

    password = input(
        "Enter password " + str(i) + ": "
    )

    result = classify_password(
        password,
        checker
    )

    print(
        "Result:",
        result
    )

print("\n----- PASSWORD AUDIT COMPLETED------")
```

Output :

```
----- PASSWORD AUDIT PROGRAM -----

Enter number of banned words: 3

Enter banned words:
Enter banned word 1: bad
Enter banned word 2: loud
Enter banned word 3: slow

Enter number of passwords to check: 1

----- ENTER PASSWORDS -----
Enter password 1: 1234
Result: WEAK_LENGTH

----- PASSWORD AUDIT COMPLETED -----
```

3. Recursive Expression Engine with Memoization

```
#Recursive Expression Engine with Memoization

import re

# Dictionary to store variable expressions
expressions = {}

# Dictionary for memoization
memo = {}

def is_valid_expression(expression):
    """Check whether the expression contains only allowed characters."""
    pattern = r'^[A-Za-z0-9_+\-*/%()\s]+$'
    return re.fullmatch(pattern, expression) is not None

def tokenize(expression):
    """Convert expression into tokens."""
    return re.findall(r'[A-Za-z_][A-Za-z0-9_]*|\d+|[()+\-*/%]', expression)

def evaluate_expression(expression, visiting):
    """Evaluate an expression recursively with cycle detection."""

    expression = expression.strip()

    # If expression is a number
    if expression.isdigit():
        return int(expression)

    # If expression is a variable
    if re.fullmatch(r'[A-Za-z_][A-Za-z0-9_]*', expression):
        return evaluate_variable(expression, visiting)

    tokens = tokenize(expression)

    if not tokens:
        raise ValueError("INVALID")

    # Handle simple expression using recursive parsing
    index = 0

    def parse_expression():
        nonlocal index

        value = parse_term()

        while index < len(tokens) and tokens[index] in ('+', '-'):
            operator = tokens[index]
            index += 1
            right = parse_term()

            if operator == '+':
                value += right
            else:
                value -= right
```

```
    return value

def parse_term():
    nonlocal index

    value = parse_factor()

    while index < len(tokens) and tokens[index] in ('*', '/', '%'):
        operator = tokens[index]
        index += 1
        right = parse_factor()

        if operator == '*':
            value *= right
        elif operator == '/':
            if right == 0:
                raise ValueError("INVALID")
            value //= right
        elif operator == '%':
            if right == 0:
                raise ValueError("INVALID")
            value %= right

    return value

def parse_factor():
    nonlocal index

    if index >= len(tokens):
        raise ValueError("INVALID")

    token = tokens[index]

    if token == '(':
        index += 1
        value = parse_expression()

        if index >= len(tokens) or tokens[index] != ')':
            raise ValueError("INVALID")

        index += 1
        return value

    if token.isdigit():
        index += 1
        return int(token)

    if re.fullmatch(r'[A-Za-z_][A-Za-z0-9_]*', token):
        index += 1
        return evaluate_variable(token, visiting)

    raise ValueError("INVALID")

result = parse_expression()

if index != len(tokens):
    raise ValueError("INVALID")
```

```
    return result

def evaluate_variable(variable, visiting):
    """Evaluate a variable and detect cyclic dependencies."""

    if variable in memo:
        return memo[variable]

    if variable not in expressions:
        raise ValueError("INVALID")

    # Cycle detection
    if variable in visiting:
        raise ValueError("CYCLE")

    visiting.add(variable)

    value = evaluate_expression(expressions[variable], visiting)

    visiting.remove(variable)

    # Store result for memoization
    memo[variable] = value

    return value

# ---------- MAIN PROGRAM ----------

print("----- RECURSIVE EXPRESSION ENGINE -----")

try:
    n = int(input("Enter number of variables: "))

    if n <= 0:
        print("Number of variables must be greater than 0.")
        exit()

    print("\nEnter variables and their expressions:")

    for i in range(n):
        print(f"\nVariable {i + 1}")
        variable = input("Enter variable name: ").strip()
        expression = input("Enter expression: ").strip()

        if not re.fullmatch(r'[A-Za-z_][A-Za-z0-9_]*', variable):
            print("Invalid variable name.")
            exit()

        if not is_valid_expression(expression):
            print("Invalid expression.")
            exit()

        expressions[variable] = expression

    print("\nEnter final expression:")
    final_expression = input("Expression: ").strip()
```

```
    if not is_valid_expression(final_expression):
        print("INVALID")
    else:
        try:
            answer = evaluate_expression(final_expression, set())
            print("\nResult:", answer)

        except ValueError as error:
            print("\n", error)

except ValueError:
    print("Please enter valid numeric input.")
```

Output :

```
----- RECURSIVE EXPRESSION ENGINE -----
Enter number of variables: 3

Enter variables and their expressions:

Variable 1
Enter variable name: a
Enter expression: 5

Variable 2
Enter variable name: b
Enter expression: a + 3

Variable 3
Enter variable name: c
Enter expression: b*2

Enter final expression:
Expression: c+8

Result: 24
```

4. Exception-Safe CSV Transaction Splitter

```
# Q4: Exception-Safe CSV Transaction Splitter

import csv
import os

def process_transactions(input_file):
    credit_file = "credit.csv"
    debit_file = "debit.csv"
    error_file = "error.csv"

    balances = {}

    try:
        with open(input_file, "r", newline="", encoding="utf-8") as file:
            reader = csv.reader(file)

            with open(credit_file, "w", newline="", encoding="utf-8") as credit:
                with open(debit_file, "w", newline="", encoding="utf-8") as debit:
                    with open(error_file, "w", newline="", encoding="utf-8") as error:

                        credit_writer = csv.writer(credit)
                        debit_writer = csv.writer(debit)
                        error_writer = csv.writer(error)

                        credit_writer.writerow(
                            ["Account", "Amount"]
                        )
                        debit_writer.writerow(
                            ["Account", "Amount"]
                        )
                        error_writer.writerow(
                            ["Row", "Reason"]
                        )

                        for row_number, row in enumerate(reader, start=1):

                            try:
                                if len(row) != 3:
                                    raise ValueError("Invalid number of fields")

                                account = row[0].strip()
                                transaction_type = row[1].strip().upper()
                                amount_text = row[2].strip()

                                if account == "":
                                    raise ValueError("Account is empty")

                                if transaction_type not in ["CREDIT", "DEBIT"]:
                                    raise ValueError(
                                        "Transaction type must be CREDIT or DEBIT"
                                    )

                                amount = float(amount_text)

                                if amount <= 0:
```

```
                                raise ValueError(
                                    "Amount must be greater than zero"
                                )

                            if account not in balances:
                                balances[account] = 0

                            if transaction_type == "CREDIT":
                                balances[account] += amount
                                credit_writer.writerow(
                                    [account, amount]
                                )

                            else:
                                balances[account] -= amount
                                debit_writer.writerow(
                                    [account, amount]
                                )

                        except ValueError as e:
                            error_writer.writerow(
                                [row_number, str(e)]
                            )

        except FileNotFoundError:
            print("Input file not found.")
            return

        except Exception as e:
            print("Error while processing file:", e)
            return

        print("\nTransaction processing completed.")
        print("\nAccount-wise Net Balance Changes:")

        sorted_balances = sorted(
            balances.items(),
            key=lambda item: abs(item[1]),
            reverse=True
        )

        for account, balance in sorted_balances:
            print(f"{account}: {balance:.2f}")

        print("\nFiles created:")
        print("credit.csv")
        print("debit.csv")
        print("error.csv")

print("CSV Transaction Splitter")

input_file = input("Enter input CSV file path: ").strip()

if not os.path.exists(input_file):
    print("File does not exist.")
else:
    process_transactions(input_file)
```

Output :

```
CSV Transaction Splitter
Enter input CSV file path: transactions.csv

Transaction processing completed.

Account-wise Net Balance Changes:
A101: 4000.00
A103: 2000.00
A102: -1000.00

Files created:
credit.csv
debit.csv
error.csv
```

5. Object-Oriented Bank Settlement System

```
# Q5: OOP Bank Settlement System

class InsufficientBalanceError(Exception):
    pass

class AccountNotFoundError(Exception):
    pass

class Account:
    def __init__(self, account_id, balance):
        self.account_id = account_id
        self._balance = balance
        self.history = []

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit amount must be greater than zero.")

        self._balance += amount
        self.history.append(f"DEPOSIT {amount}")

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("Withdraw amount must be greater than zero.")

        if amount > self._balance:
            raise InsufficientBalanceError("Insufficient balance.")

        self._balance -= amount
        self.history.append(f"WITHDRAW {amount}")

class Transaction:
    def __init__(self, transaction_type, account1, account2=None, amount=0):
        self.transaction_type = transaction_type
        self.account1 = account1
        self.account2 = account2
        self.amount = amount

class Bank:
    def __init__(self):
        self.accounts = {}

    def add_account(self, account):
        self.accounts[account.account_id] = account

    def get_account(self, account_id):
        if account_id not in self.accounts:
            raise AccountNotFoundError("Account not found.")

        return self.accounts[account_id]
```

```
def deposit(self, account_id, amount):
    account = self.get_account(account_id)
    account.deposit(amount)

def withdraw(self, account_id, amount):
    account = self.get_account(account_id)
    account.withdraw(amount)

def transfer(self, from_id, to_id, amount):
    sender = self.get_account(from_id)
    receiver = self.get_account(to_id)

    if amount <= 0:
        raise ValueError("Transfer amount must be greater than zero.")

    if amount > sender.balance:
        raise InsufficientBalanceError("Insufficient balance.")

    sender._balance -= amount
    receiver._balance += amount

    sender.history.append(
        f"TRANSFER TO {to_id} {amount}"
    )

    receiver.history.append(
        f"TRANSFER FROM {from_id} {amount}"
    )

def execute_transaction(self, transaction):
    if transaction.transaction_type == "DEPOSIT":
        self.deposit(
            transaction.account1,
            transaction.amount
        )

    elif transaction.transaction_type == "WITHDRAW":
        self.withdraw(
            transaction.account1,
            transaction.amount
        )

    elif transaction.transaction_type == "TRANSFER":
        self.transfer(
            transaction.account1,
            transaction.account2,
            transaction.amount
        )

    else:
        raise ValueError("Invalid transaction type.")

def execute_batch(self, transactions):
    old_balances = {
        account_id: account.balance
        for account_id, account in self.accounts.items()
    }
```

```
        old_history = {
            account_id: list(account.history)
            for account_id, account in self.accounts.items()
        }

        try:
            for transaction in transactions:
                self.execute_transaction(transaction)

        except Exception:
            for account_id, balance in old_balances.items():
                self.accounts[account_id]._balance = balance
                self.accounts[account_id].history = old_history[
                    account_id
                ]

            raise

print("Bank Settlement System")

bank = Bank()

try:
    n = int(input("Enter number of accounts: "))

    if n <= 0:
        print("Number of accounts must be greater than zero.")
        exit()

    print("\nEnter account details:")

    for i in range(n):
        print(f"\nAccount {i + 1}")

        account_id = input("Enter account ID: ").strip()
        balance = float(input("Enter initial balance: "))

        if balance < 0:
            print("Balance cannot be negative.")
            exit()

        bank.add_account(Account(account_id, balance))

    t = int(input("\nEnter number of transactions: "))

    print("\nEnter transactions:")
    print("Format:")
    print("DEPOSIT account amount")
    print("WITHDRAW account amount")
    print("TRANSFER from_account to_account amount")

    transactions = []

    for i in range(t):
        print(f"\nTransaction {i + 1}")
        parts = input("Enter transaction: ").split()
```

```
    if len(parts) == 3:
        transaction_type = parts[0].upper()
        account1 = parts[1]
        amount = float(parts[2])

        transactions.append(
            Transaction(
                transaction_type,
                account1,
                amount=amount
            )
        )

    elif len(parts) == 4:
        transaction_type = parts[0].upper()
        account1 = parts[1]
        account2 = parts[2]
        amount = float(parts[3])

        transactions.append(
            Transaction(
                transaction_type,
                account1,
                account2,
                amount
            )
        )

    else:
        print("Invalid transaction format.")
        exit()

batch_choice = input(
    "\nExecute all transactions as one batch? (yes/no): "
).strip().lower()

if batch_choice == "yes":

    try:
        bank.execute_batch(transactions)
        print("\nAll transactions completed successfully.")

    except Exception as e:
        print("\nFAILED")
        print("Reason:", str(e))

else:

    for i, transaction in enumerate(transactions, start=1):
        try:
            bank.execute_transaction(transaction)
            print(f"Transaction {i}: SUCCESS")

        except Exception as e:
            print(f"Transaction {i}: FAILED")
            print("Reason:", str(e))
```

```
    print("\nFinal Account Balances:")

    for account_id, account in bank.accounts.items():
        print(f"{account_id}: {account.balance:.2f}")

except ValueError:
    print("Please enter valid input.")
```

Output :

```
Bank Settlement System
Enter number of accounts: 1

Enter account details:

Account 1
Enter account ID: 1
Enter initial balance: 1000

Enter number of transactions: 2

Enter transactions:
Format:
DEPOSIT account amount
WITHDRAW account amount
TRANSFER from_account to_account amount

Transaction 1
Enter transaction: DEPOSIT 1 1000

Transaction 2
Enter transaction: WITHDRAW 1 500

Execute all transactions as one batch? (yes/no): YES

All transactions completed successfully.

Final Account Balances:
1: 1500.00
```

6. Python Module Dependency Resolver

```
# Q6: Module Dependency Resolver

from collections import defaultdict, deque

def find_cycle(modules, graph, indegree):
    """Find a cycle when topological sorting is not possible."""

    visited = set()
    path = []
    position = {}

    def dfs(module):
        visited.add(module)
        position[module] = len(path)
        path.append(module)

        for next_module in graph[module]:
            if next_module not in visited:
                cycle = dfs(next_module)
                if cycle:
                    return cycle

            elif next_module in position:
                start = position[next_module]
                return path[start:] + [next_module]

        path.pop()
        position.pop(module)
        return None

    for module in modules:
        if module not in visited:
            cycle = dfs(module)
            if cycle:
                return cycle

    return []

def resolve_dependencies(modules, dependencies):
    """Perform lexicographically smallest topological sorting."""

    graph = defaultdict(list)
    indegree = {module: 0 for module in modules}

    for a, b in dependencies:
        # b must be loaded before a
        graph[b].append(a)
        indegree[a] += 1

    # Lexicographically smallest module first
    ready = []
    import heapq

    for module in modules:
        if indegree[module] == 0:
```

```
            heapq.heappush(ready, module)

    order = []

    while ready:
        current = heapq.heappop(ready)
        order.append(current)

        for next_module in graph[current]:
            indegree[next_module] -= 1

            if indegree[next_module] == 0:
                heapq.heappush(ready, next_module)

    if len(order) == len(modules):
        return order, None

    cycle = find_cycle(modules, graph, indegree)
    return None, cycle

print("Module Dependency Resolver")

try:
    n = int(input("Enter number of modules: "))

    if n <= 0:
        print("Number of modules must be greater than zero.")
        exit()

    modules = []

    print("\nEnter module names:")

    for i in range(n):
        module = input(f"Enter module {i + 1}: ").strip()

        if module == "":
            print("Module name cannot be empty.")
            exit()

        if module in modules:
            print("Duplicate module name is not allowed.")
            exit()

        modules.append(module)

    e = int(input("\nEnter number of dependencies: "))

    if e < 0:
        print("Number of dependencies cannot be negative.")
        exit()

    dependencies = []

    print("\nEnter dependencies.")
    print("If module A imports module B, enter: A B")
```

```
    for i in range(e):
        print(f"\nDependency {i + 1}")

        parts = input("Enter dependency: ").split()

        if len(parts) != 2:
            print("Invalid dependency format.")
            exit()

        a, b = parts

        if a not in modules or b not in modules:
            print("Both modules must exist.")
            exit()

        dependencies.append((a, b))

    order, cycle = resolve_dependencies(modules, dependencies)

    if cycle:
        print("\nCYCLE")
        print("Cycle:", " -> ".join(cycle))
    else:
        print("\nValid Loading Order:")
        print(" -> ".join(order))

except ValueError:
    print("Please enter valid numeric input.")
```

Output :

```
Module Dependency Resolver
Enter number of modules: 3

Enter module names:
Enter module 1: A
Enter module 2: B
Enter module 3: C

Enter number of dependencies: 3

Enter dependencies.
If module A imports module B, enter: A B

Dependency 1
Enter dependency: A B

Dependency 2
Enter dependency: B C

Dependency 3
Enter dependency: C A

CYCLE
Cycle: A -> C -> B -> A
```

7. Interactive Formula Validator with Custom Exceptions

```
# Q7: Interactive Formula Validator with Custom Exceptions

class InvalidFormatError(Exception):
    pass

class UnknownVariableError(Exception):
    pass

class DivisionByZeroError(Exception):
    pass

class UnsupportedOperatorError(Exception):
    pass

variables = {}

def evaluate_formula(formula):
    formula = formula.strip()

     if "**" in formula or "//" in formula:
         raise UnsupportedOperatorError(
             "Unsupported operator used."
         )

    allowed_operators = ["+", "-", "*", "/", "%"]

    found_operator = False

    for operator in allowed_operators:
        if operator in formula:
            found_operator = True
            break

    if not found_operator:
        raise InvalidFormatError(
            "Invalid formula format."
        )

    parts = formula.split()

    if len(parts) != 3:
        raise InvalidFormatError(
            "Formula must be in the format: operand operator operand"
        )

    left = parts[0]
    operator = parts[1]
    right = parts[2]

     if operator not in allowed_operators:
         raise UnsupportedOperatorError(
             "Unsupported operator used."
         )

    def get_value(value):
```

```
            try:
                return int(value)
            except ValueError:
                if value not in variables:
                    raise UnknownVariableError(
                        f"Unknown variable: {value}"
                    )
                return variables[value]

        left_value = get_value(left)
        right_value = get_value(right)

        if operator == "+":
            return left_value + right_value

        elif operator == "-":
            return left_value - right_value

        elif operator == "*":
            return left_value * right_value

        elif operator == "/":
            if right_value == 0:
                raise DivisionByZeroError(
                    "Cannot divide by zero."
                )
            return left_value / right_value

        elif operator == "%":
            if right_value == 0:
                raise DivisionByZeroError(
                    "Cannot perform modulo by zero."
                )

        raise UnsupportedOperatorError(
            "Unsupported operator used."
        )

print("Interactive Formula Validator")
print("Enter 'quit' to stop the program.")

while True:

    try:
        user_input = input("\nEnter formula: ").strip()

        if user_input.lower() == "quit":
            print("Program terminated.")
            break

        if "=" in user_input:
            parts = user_input.split("=")

            if len(parts) != 2:
                raise InvalidFormatError(
                    "Invalid assignment format."
                )
```

```
            variable = parts[0].strip()
            value = parts[1].strip()

            if not variable.isidentifier():
                raise InvalidFormatError(
                    "Invalid variable name."
                )

            try:
                variables[variable] = int(value)
                print(f"{variable} = {variables[variable]}")

            except ValueError:
                raise InvalidFormatError(
                    "Variable value must be an integer."
                )

        else:
            result = evaluate_formula(user_input)
            print("Result:", result)

    except InvalidFormatError as e:
        print("InvalidFormatError:", e)

    except UnknownVariableError as e:
        print("UnknownVariableError:", e)

    except DivisionByZeroError as e:
        print("DivisionByZeroError:", e)

    except UnsupportedOperatorError as e:
        print("UnsupportedOperatorError:", e)
```

Output :

```
Interactive Formula Validator
Enter 'quit' to stop the program.

Enter formula: x = 10
x = 10

Enter formula: x+5
InvalidFormatError: Formula must be in the format: operand operator ope

Enter formula: x + 5
Result: 15

Enter formula: x - 20
Result: -10

Enter formula: quit
Program terminated.
```

8. Compressed Log Index using Pickle and Zip

```
# Q8: Compressed Log Index using Pickle and Zip

import os
import re
import pickle
import zipfile

def build_index(folder_path, output_zip):
    index = {}

    if not os.path.isdir(folder_path):
        print("Folder does not exist.")
        return

    log_files = []

    for root, dirs, files in os.walk(folder_path):
        for file in files:
            if file.lower().endswith(".txt"):
                log_files.append(os.path.join(root, file))

    if not log_files:
        print("No text log files found.")
        return

    for file_path in log_files:
        try:
            with open(file_path, "r", encoding="utf-8") as file:
                for line_number, line in enumerate(file, start=1):

                    # Convert the line into normalized words
                    words = re.findall(r"[A-Za-z0-9_]+", line.lower())

                    for word in words:
                        if word not in index:
                            index[word] = {}

                        file_name = os.path.basename(file_path)

                        if file_name not in index[word]:
                            index[word][file_name] = []

                        if line_number not in index[word][file_name]:
                            index[word][file_name].append(line_number)

        except Exception as e:
            print("Error reading", file_path, ":", e)

    index_file = os.path.join(folder_path, "log_index.pkl")

    try:
        with open(index_file, "wb") as file:
            pickle.dump(index, file)

        with zipfile.ZipFile(
```

```
            output_zip, "w", zipfile.ZIP_DEFLATED
        ) as zip_file:

            for file_path in log_files:
                zip_file.write(
                    file_path,
                    arcname=os.path.basename(file_path)
                )

            zip_file.write(
                index_file,
                arcname="log_index.pkl"
            )

        print("\nIndex created successfully.")
        print("Compressed file:", output_zip)

    except Exception as e:
        print("Error creating index:", e)

def search_index(index_file, query):
    if not os.path.exists(index_file):
        print("Index file does not exist.")
        return

    try:
        with open(index_file, "rb") as file:
            index = pickle.load(file)

        tokens = re.findall(r"[A-Za-z0-9_]+", query.lower())

        if not tokens:
            print("Invalid search query.")
            return

        found = False

        for token in tokens:

            if token in index:
                found = True
                print(f"\nMatches for '{token}':")

                for file_name, lines in index[token].items():
                    for line_number in lines:
                        print(
                            f"{file_name}:{line_number}"
                        )
            else:
                print(f"\nNo matches for '{token}'.")

        if not found:
            print("\nNo matching tokens found.")

    except Exception as e:
        print("Error searching index:", e)
```

```
print("Compressed Log Index")

print("\n1. BUILD")
print("2. SEARCH")

choice = input("\nEnter your choice: ").strip()

if choice == "1":

    folder_path = input(
        "Enter log folder path: "
    ).strip()

    output_zip = input(
        "Enter output ZIP file name: "
    ).strip()

    if output_zip == "":
        output_zip = "compressed_logs.zip"

    build_index(folder_path, output_zip)

elif choice == "2":

    index_file = input(
        "Enter pickle index file path: "
    ).strip()

    query = input(
        "Enter search query: "
    ).strip()

    search_index(index_file, query)

else:
    print("Invalid choice.")
```

Output :

```
Compressed Log Index

1. BUILD
2. SEARCH

Enter your choice: 1
Enter log folder path: logs
Enter output ZIP file name: logs.zip
Folder does not exist.
```

9. Threaded Job Scheduler Simulation

```
# Q9: Threaded Job Scheduler Simulation

import threading
import heapq

class Job:
    def __init__(self, arrival, job_id, priority, duration, resources):
        self.arrival = arrival
        self.job_id = job_id
        self.priority = priority
        self.duration = duration
        self.resources = resources

def schedule_jobs(workers, jobs):
    # Sort jobs by arrival time first
    jobs.sort(key=lambda job: job.arrival)

    worker_available = [0] * workers
    results = []

    current_time = 0
    waiting_time = 0

    lock = threading.Lock()

    # Process jobs according to arrival and priority
    pending = []
    index = 0

    while index < len(jobs) or pending:

        if not pending and index < len(jobs):
            current_time = max(current_time, jobs[index].arrival)

        while index < len(jobs) and jobs[index].arrival <= current_time:
            job = jobs[index]

            # Higher priority first
            heapq.heappush(
                pending,
                (-job.priority, job.arrival, job.job_id, job)
            )

            index += 1

        if not pending:
            continue

        with lock:
            _, _, _, job = heapq.heappop(pending)

            # Select worker that becomes available earliest
            worker_id = min(
                range(workers),
                key=lambda i: worker_available[i]
```

```
            )

            start_time = max(
                job.arrival,
                worker_available[worker_id]
            )

            finish_time = start_time + job.duration

            wait = start_time - job.arrival
            waiting_time += wait

            worker_available[worker_id] = finish_time

            results.append(
                (
                    job,
                    worker_id + 1,
                    start_time,
                    finish_time
                )
            )

            current_time = start_time

    results.sort(key=lambda item: item[2])

    return results, waiting_time

print("Threaded Job Scheduler Simulation")

try:
    first_line = input(
        "Enter number of workers and jobs: "
    ).split()

    if len(first_line) != 2:
        print("Please enter two numbers.")
        exit()

    w = int(first_line[0])
    n = int(first_line[1])

    if w <= 0 or n <= 0:
        print("Workers and jobs must be greater than zero.")
        exit()

    jobs = []

    print("\nEnter job details:")
    print("Format: arrival job_id priority duration resources")

    for i in range(n):
        print(f"\nJob {i + 1}")

        parts = input("Enter job details: ").split()
```

```
        if len(parts) != 5:
            print("Invalid job format.")
            exit()

        arrival = int(parts[0])
        job_id = parts[1]
        priority = int(parts[2])
        duration = int(parts[3])
        resources = int(parts[4])

        if arrival < 0:
            print("Arrival time cannot be negative.")
            exit()

        if duration <= 0:
            print("Duration must be greater than zero.")
            exit()

        if resources <= 0:
            print("Resources must be greater than zero.")
            exit()

        jobs.append(
            Job(
                arrival,
                job_id,
                priority,
                duration,
                resources
            )
        )

    results, total_waiting = schedule_jobs(w, jobs)

    print("\nJob Schedule:")

    for job, worker, start, finish in results:
        print(
            f"{job.job_id} W{worker} "
            f"{start} {finish}"
        )

    average_wait = total_waiting / n

    print(f"AVG_WAIT {average_wait:.2f}")

except ValueError:
    print("Please enter valid numeric values.")
```

Output :

```
Threaded Job Scheduler Simulation
Enter number of workers and jobs: 2 3

Enter job details:
Format: arrival job_id priority duration resources

Job 1
Enter job details: 0 31 2 5 1

Job 2
Enter job details: 1 32 5 3 1

Job 3
Enter job details: 2 32 2 2 1

Job Schedule:
31 W1 0 5
32 W2 1 4
32 W2 4 6
AVG_WAIT 0.67
```

10. Tkinter Assignment Tracker with File Persistence

```
# Q10: Tkinter Assignment Tracker with JSON/CSV Persistence

import tkinter as tk
from tkinter import ttk, messagebox, filedialog
import json
import csv
import os

DATA_FILE = "assignments.json"

class AssignmentTracker:
    def __init__(self, root):
        self.root = root
        self.root.title("Assignment Tracker")
        self.root.geometry("750x550")

        self.data = []

        self.load_data()
        self.create_widgets()
        self.refresh_table()

    def load_data(self):
        if os.path.exists(DATA_FILE):
            try:
                with open(DATA_FILE, "r") as file:
                    self.data = json.load(file)
            except:
                self.data = []

    def save_data(self):
        with open(DATA_FILE, "w") as file:
            json.dump(self.data, file, indent=4)

    def create_widgets(self):

        title = tk.Label(
            self.root,
            text="Assignment Tracker",
            font=("Arial", 20, "bold")
        )
        title.pack(pady=10)

        frame = tk.Frame(self.root)
        frame.pack(pady=10)

        tk.Label(frame, text="Enrollment No:").grid(
            row=0, column=0, padx=5, pady=5
        )

        self.enrollment_entry = tk.Entry(frame)
        self.enrollment_entry.grid(
            row=0, column=1, padx=5, pady=5
        )
```

```
tk.Label(frame, text="Student Name:").grid(
    row=1, column=0, padx=5, pady=5
)

self.name_entry = tk.Entry(frame)
self.name_entry.grid(
    row=1, column=1, padx=5, pady=5
)

tk.Label(frame, text="Assignment:").grid(
    row=2, column=0, padx=5, pady=5
)

self.assignment_entry = tk.Entry(frame)
self.assignment_entry.grid(
    row=2, column=1, padx=5, pady=5
)

tk.Label(frame, text="Marks:").grid(
    row=3, column=0, padx=5, pady=5
)

self.marks_entry = tk.Entry(frame)
self.marks_entry.grid(
    row=3, column=1, padx=5, pady=5
)

tk.Label(frame, text="Total Marks:").grid(
    row=4, column=0, padx=5, pady=5
)

self.total_entry = tk.Entry(frame)
self.total_entry.grid(
    row=4, column=1, padx=5, pady=5
)

tk.Label(frame, text="Status:").grid(
    row=5, column=0, padx=5, pady=5
)

self.status = ttk.Combobox(
    frame,
    values=["Pending", "Completed"],
    state="readonly"
)
self.status.set("Pending")
self.status.grid(
    row=5, column=1, padx=5, pady=5
)

button_frame = tk.Frame(self.root)
button_frame.pack(pady=10)

tk.Button(
    button_frame,
    text="Add Submission",
    command=self.add_submission
```

```
    ).grid(row=0, column=0, padx=5)

    tk.Button(
        button_frame,
        text="Update Marks",
         command=self.update_marks
    ).grid(row=0, column=1, padx=5)

    tk.Button(
        button_frame,
        text="Filter",
        command=self.filter_data
    ).grid(row=0, column=2, padx=5)

    tk.Button(
        button_frame,
        text="Show All",
        command=self.refresh_table
    ).grid(row=0, column=3, padx=5)

    tk.Button(
        button_frame,
        text="Export CSV",
        command=self.export_csv
    ).grid(row=0, column=4, padx=5)

    columns = (
        "Enrollment",
        "Name",
        "Assignment",
        "Marks",
        "Total",
        "Status"
    )

    self.table = ttk.Treeview(
        self.root,
        columns=columns,
        show="headings"
    )

    for column in columns:
        self.table.heading(column, text=column)
        self.table.column(column, width=110)

    self.table.pack(
        fill="both",
        expand=True,
        padx=10,
        pady=10
    )

def add_submission(self):

    enrollment = self.enrollment_entry.get().strip()
    name = self.name_entry.get().strip()
    assignment = self.assignment_entry.get().strip()
```

```
        marks = self.marks_entry.get().strip()
        total = self.total_entry.get().strip()
        status = self.status.get()

        if not enrollment or not name or not assignment:
            messagebox.showerror(
                "Error",
                "Please enter all required details."
            )
            return

        try:
            marks = float(marks)
            total = float(total)

            if marks < 0 or total <= 0 or marks > total:
                raise ValueError

        except ValueError:
            messagebox.showerror(
                "Error",
                "Enter valid marks."
            )
            return

        record = {
            "enrollment": enrollment,
            "name": name,
            "assignment": assignment,
            "marks": marks,
            "total": total,
            "status": status
        }

        self.data.append(record)
        self.save_data()
        self.refresh_table()

        messagebox.showinfo(
            "Success",
            "Submission added successfully."
        )

    def update_marks(self):

        selected = self.table.selection()

        if not selected:
            messagebox.showerror(
                "Error",
                "Select a submission first."
            )
            return

        item = self.table.item(selected[0])
        values = item["values"]
```

```
    enrollment = values[0]
    assignment = values[2]

    try:
        new_marks = float(self.marks_entry.get())

    except ValueError:
        messagebox.showerror(
            "Error",
            "Enter valid marks."
        )
        return

    for record in self.data:

        if (
            record["enrollment"] == enrollment
            and record["assignment"] == assignment
        ):
            if new_marks < 0 or new_marks > record["total"]:
                messagebox.showerror(
                    "Error",
                    "Marks are out of range."
                )
                return

            record["marks"] = new_marks

            if new_marks > 0:
                record["status"] = "Completed"

            break

    self.save_data()
    self.refresh_table()

    messagebox.showinfo(
        "Success",
        "Marks updated successfully."
    )

def refresh_table(self):

    for item in self.table.get_children():
        self.table.delete(item)

    for record in self.data:
        self.table.insert(
            "",
            "end",
            values=(
                record["enrollment"],
                record["name"],
                record["assignment"],
                record["marks"],
                record["total"],
                record["status"]
```

```
            )
        )

def filter_data(self):

    status = self.status.get()

    for item in self.table.get_children():
        self.table.delete(item)

    for record in self.data:
        if record["status"] == status:
            self.table.insert(
                "",
                "end",
                values=(
                    record["enrollment"],
                    record["name"],
                    record["assignment"],
                    record["marks"],
                    record["total"],
                    record["status"]
                )
            )

def export_csv(self):

    if not self.data:
        messagebox.showerror(
            "Error",
            "No data available to export."
        )
        return

    file_path = filedialog.asksaveasfilename(
        defaultextension=".csv",
        filetypes=[("CSV Files", "*.csv")]
    )

    if not file_path:
        return

    with open(
        file_path,
        "w",
        newline="",
        encoding="utf-8"
    ) as file:

        writer = csv.DictWriter(
            file,
            fieldnames=[
                "enrollment",
                "name",
                "assignment",
                "marks",
                "total",
```

```
                "status"
            ]
        )

        writer.writeheader()
        writer.writerows(self.data)

    messagebox.showinfo(
        "Success",
        "Data exported successfully."
    )

root = tk.Tk()
app = AssignmentTracker(root)
root.mainloop()
```

Output :

